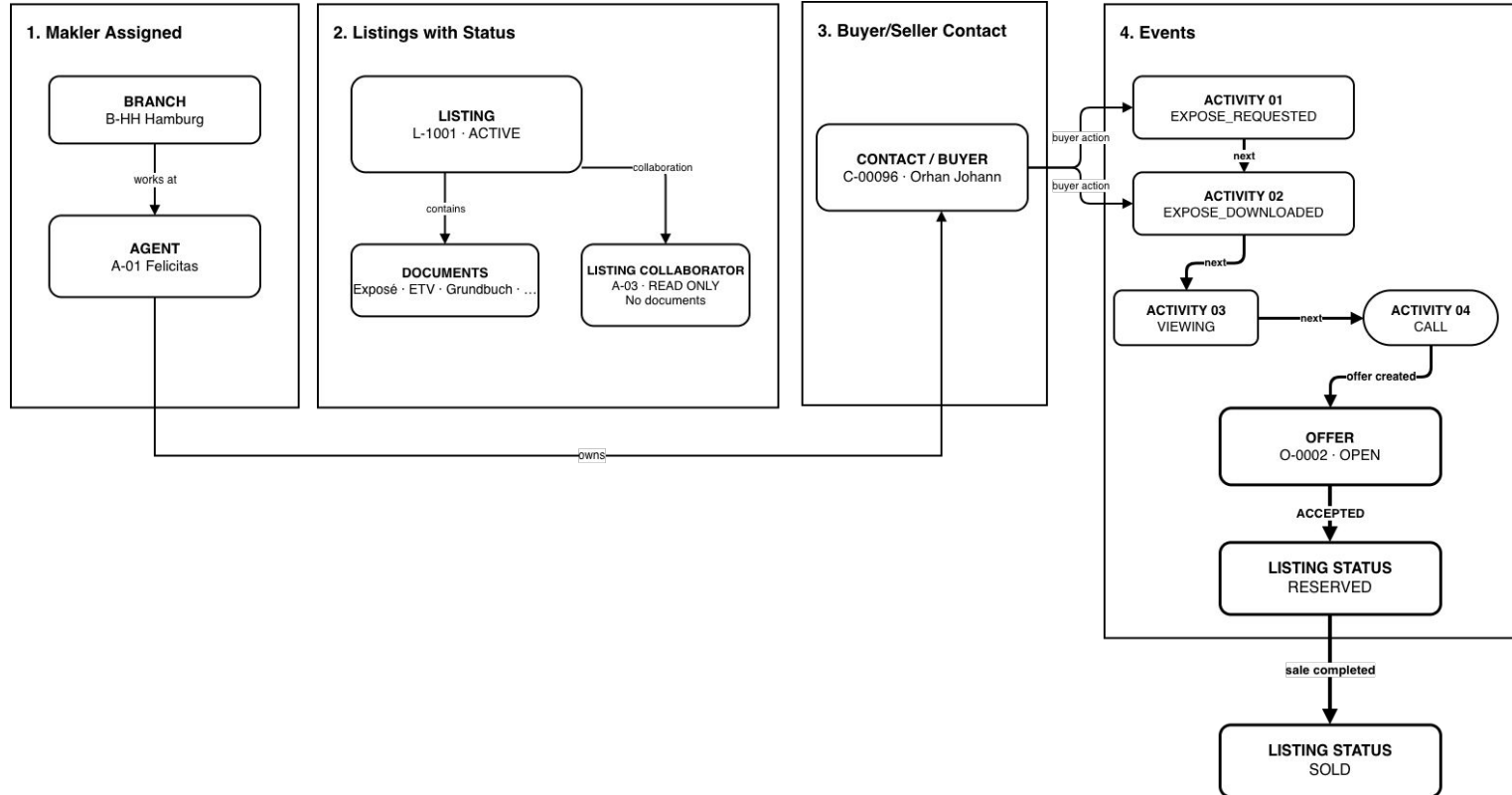


# Evernest AI Case Study

By: Sachit Bhavsar

# Understanding Business Logic



# Problems Identified with existing data flow

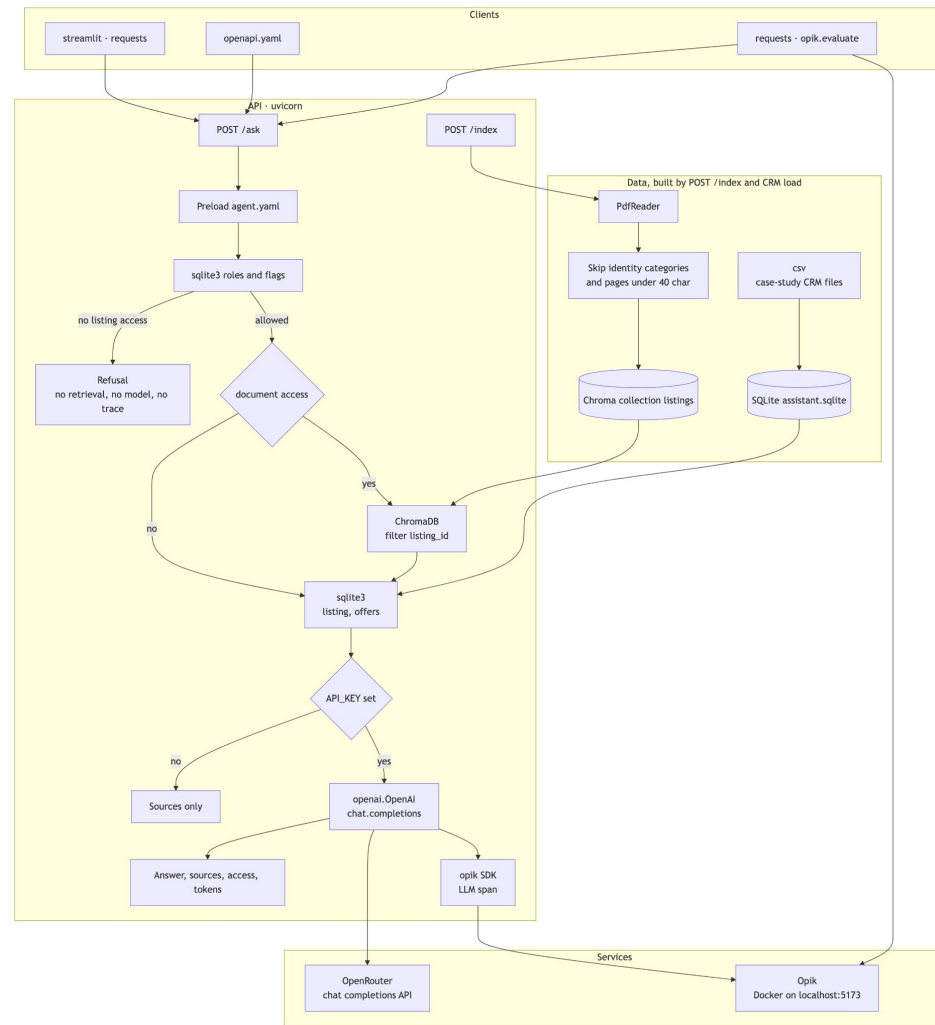
| Issue Identified                                                                             | Example                                                             | Investigated                                                                                            | Potential Fix                                                                                  |
|----------------------------------------------------------------------------------------------|---------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------|
| One contact email and several mekler may work with them                                      | The same normalised email has more than one <code>contact_id</code> | Birthe Keudel is <code>C-00108</code> owned by A-01 and <code>C-00402</code> owned by A-05              | Unique index on the normalised email. The second mekler adds a relationship, not a new contact |
| An activity event (call or an exposé request) belongs to a listing that exists               | <code>listing_id</code> is empty or it is not in listings           | Multiple calls by A-03 with no listing. Fail on <code>listing_id</code> that does not join to listings. | FK on <code>listing_id</code> then an activity will require a listing reference                |
| A home is a listing row, so the same flat is inserted twice and each copy gets its own offer | <code>L-1009</code> and <code>L-1017</code> both are reserved       | The same normalised street and zip has two <code>ACTIVE</code> or <code>RESERVED</code> listings.       | A unit, unique on address. blocks a second live listing on the same unit.                      |

# Continue with data flow

| Issue Identified                                                                                                                                 | Example                                                                                                                                                              | Investigated                                                                                                              | Potential Fix                                                                                     |
|--------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
| An offer is just an amount. It is not tied to the asking price or to the previous offer from that buyer. Offer is not a bid on the previous bid. | <code>O-0005</code> is 530.000 € and later <code>O-0006</code> is 5.300.000 €, same contact, both open.<br><br>One open bid per buyer, in range of the asking price. | <code>amount_eur</code> is just a number. A second <code>OPEN</code> row does not close the first.                        | A new bid from the same party on the same listing replaces the open one.                          |
| Start and completion are two strings, with no rule that completion comes after the start                                                         | <code>completed_at</code> is earlier than <code>activity_at</code> , or the timestamp has no single format.                                                          | One <code>timestampz</code> , stored in UTC. <code>completed_at</code> is written only when that action actually finishes | Parse only a declared format. Fail anything else. Fail <code>completed_at &lt; activity_at</code> |

# Proposed Architecture

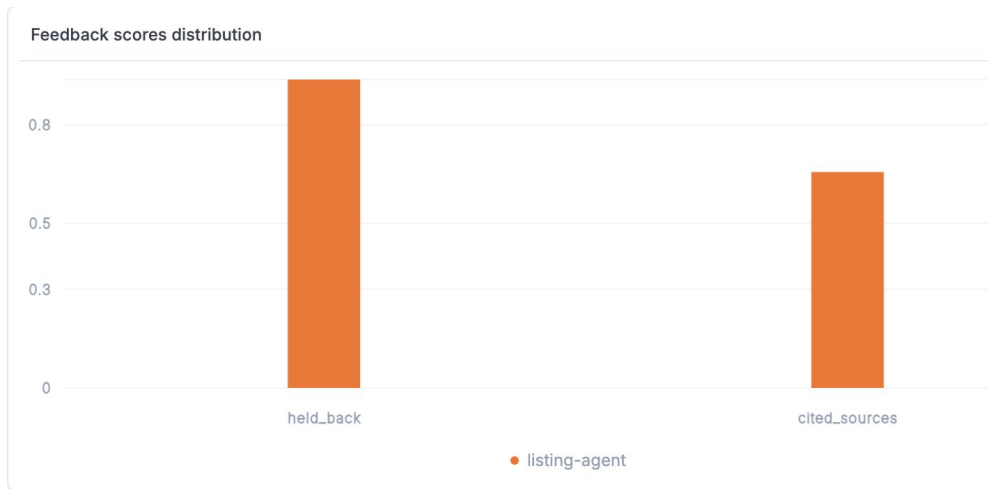
- A new client only needs the schema. The response always carries the answer, the sources, and the access decision.
- The schema stays still while the prompt, the model, and the host change. A page written against this file keeps working after a retune.
- The LLM agent behavior sits in agent.yaml. The API reloads that file on every request, so a prompt or sampling change is a file edit.
- A single call can still override the model, temperature etc. The system prompt stays in the spec, so one request can try another model without rewriting the rules.
- Access runs on SQLite before Chroma is queried and before the model client opens. A caller with no listing access gets a refusal, empty sources, and no trace.
- Documents and CRM stay in different stores. The answer can cite a PDF page and a CRM row as separate sources.



# Evaluation Metrics

Ran across the dataset as experiment,

- **cited\_sources (0.66):** whether the document ids the question expected, e.g. how many came back on the answer? 1 means all of them. 0.5 means half. 0 means none.
- **held\_back (0.94):** whether the agent stayed inside access rules. For a normal question the score is 1 and does not judge the answer. It drops only when a blocked listing was answered anyway.



# Next Steps (Recommendation)

- Bind `agent_id` to the caller. Currently, the body chooses the identity and `/ask` and `/index` accept any client. A header or token that maps to one agent id makes the SQLite access check something the client cannot rewrite.
- **Intent Router:** The "Gatekeeper" that classifies queries into SQL, Vector, or Hybrid paths using the Data Profile to prevent hallucinations.
- **Analytical Execution:** Native streaming via DuckDB (no local file downloads) for high-speed quantitative analysis.
- **Semantic Execution:** Similarity search using ChromaDB and OpenRouter embeddings for qualitative context.
- **Context Pruning:** Smart truncation (e.g., top 15 rows) to maximize token efficiency and response grounding.

